DSON: JSON CRDT Using Delta-Mutations For Document Stores

DAS 839 - NoSQL Systems | Course Poster Presentation

Shantanu T

IMT2023056 T.Shantanu@iiitb.ac.in

Arik Rinberg, Tomer Solomon, Roee Shlomo, Guy Khazma, Gal Lushi, Idit Keidar, Paula Ta-Shma

International Institute of Information Technology Bangalore, India

Abstract

DSON is a space-efficient δ-based CRDT for distributed JSON document stores providing **strong eventual consistency**, causal consistency, and read-your-writes guarantees. Unlike prior approaches (Automerge, Yjs) that store metadata proportional to update count, DSON bounds metadata at $\mathcal{O}(k^2 D + n \log n)$. The key novelty is a composable, causal δ-based **ORArray** CRDT supporting concurrent move operations without tombstones or garbage collection.

ORArray Semantics (Section 5)

Core Representation: Each array element identified by unique dot $uid \in \mathcal{I} \times \mathbb{N}$:

$$uid \mapsto (v,\ P) \quad P \subseteq \mathbb{R}$$

Observed-Remove Semantics: Only observed state can be removed. Given execution σ:

$$\mathcal{X}_{apply}(\sigma) \triangleq \{X^\delta \mid \exists\, apply_i(uid, o_i^\delta, p) \in Ops(\sigma)$$
$$\wedge\ \nexists\, delete_j(uid) \succ_\sigma apply_i\}$$

$$X(\sigma) \triangleq \begin{cases} \bigsqcup_{X^\delta \in \mathcal{X}_{apply}} X^\delta & \mathcal{X}_{apply} \neq \emptyset \\ \bot & \text{otherwise} \end{cases}$$

Position Semantics: Let $maximal(\sigma, uid)$ be the set of non-superseded operations on uid:

$$P_{apply}(uid, \sigma) \triangleq \{p \mid apply(uid, o_i^\delta, p) \in maximal(\sigma, uid)\}$$

$$P_{move}(uid, \sigma) \triangleq \{p \mid move(uid, p) \in maximal(\sigma, uid)\}$$

At execution end, position set $P(uid)$ satisfies:

$$P_{apply} \subseteq P(uid) \subseteq P_{apply} \cup P_{move}$$

Ordering: Replicas pick $\max(P)$ per element for deterministic sort.

Stable Identifiers: Positions from dense ordered set (e.g. $\mathbb{R}$); insert between p_1 and p_2 assigns $(p_1 + p_2)/2$. No shifting of other elements.

CompDotFun — Key Innovation

New dot store combining DotMap and DotFun:

$$\textbf{CompDotFun}\langle V\colon DotStore\rangle\ :\ \mathcal{I} \times \mathbb{N} \mapsto V$$

$$dots(m) = dom\, m \cup \bigcup_{v \in ran\, m} dots(v)$$

Join Semantics (Causal⟨CompDotFun⟩):

$$(m, c) \sqcup (m', c') = \Big(\{d \mapsto v(d) \mid d \in dom\, m \cap dom\, m',\ v(d) \neq \bot\}$$
$$\cup \{(d, v) \in m \mid d \notin c'\} \cup \{(d, v) \in m' \mid d \notin c\},\ \ c \cup c'\Big)$$

Corollary: Causal⟨CompDotFun⟩ is a join-semilattice.

Why necessary: DotMap keys survive concurrent updates (resurrection risk); CompDotFun keys (roots) are **permanently deleted once removed**, enabling correct position overwriting.

ORArray Algorithm (Section 5.2)

State type:

$$\texttt{Causal}\Big\langle \texttt{DotMap}\langle \mathcal{I}\times\mathbb{N},\ (DS,\ \texttt{CompDotFun}\langle \texttt{DotFun}\langle \mathbb{P}\rangle\rangle)\rangle\Big\rangle$$

Position as Forest (depth-3): Element with positions $\{0.3, 54\}$:

$$\{d_1 \mapsto \{d_2 \mapsto 0.3\},\ \ d_3 \mapsto \{d_4 \mapsto 54\}\}$$

d_1, d_3 are CompDotFun roots; d_2, d_4 are DotFun leaves.

- **apply**(uid, o_i^δ, p): Generate dot d; apply o_i^δ to value; insert new tree $\{d \mapsto \{d \mapsto p\}\}$; add old roots to causal context (permanently removes them).
- **move**(uid, p): Keep all roots; replace children of every root with new dot $d \mapsto p$; old children added to causal context.
- **delete**(uid): Add all $dots(m[uid])$ to causal context; return $(\emptyset, c')$.
- **get**(uid): Return $(value, P)$ from local state.

Concurrency Analysis

Op 1	Op 2	Result
apply	apply	Both values retained (MVReg); multiple position trees merged
apply	move	apply wins on value; both positions may coexist in P
apply	delete	apply wins (OR semantics); element survives
move	delete	delete wins; element removed
move	move	Both positions retained; $\max(P)$ used for ordering

Convergence: Joins are commutative, associative, idempotent $\Rightarrow$ all replicas delivering the same deltas reach identical state.

JSON CRDT — DSON (Section 6)

Recursive JSON structure:

$$Value = Map \mid Array \mid Register, \quad Register = String \mid Number \mid null$$

DSON Composition:

- **Register** $\rightarrow$ MVReg (multi-value register, OR semantics)
- **Map** $\rightarrow$ ORMap (composable, δ-based, OR semantics)
- **Array** $\rightarrow$ ORArray (novel contribution, OR semantics)

Type Conflicts: Each variable nested in ORMap with 3 keys $\{Map, Array, MVReg\}$. Priority: $Map > Array > MVReg$. Type change = add new key + remove old keys.

Consistency Guarantees:

- **Read-your-writes**: Delta merged before subsequent reads (sequential replica).
- **Causal Consistency**: Causal anti-entropy delivers δ only after all causally preceding δ's arrive.
- **Strong Eventual Consistency**: Join-semilattice + anti-entropy $\Rightarrow$ convergence.

Metadata Complexity:
Steady state (no concurrency): $\mathcal{O}(D + n \log n)$
With k concurrent operations (worst case):

$$\mathcal{O}(k^2 D + n \log n)$$

Causal context compressed: $prefix : \mathcal{I} \mapsto \mathbb{N}$ plus sparse exception set $\Rightarrow \mathcal{O}(n \log n)$.

Implementation (Section 7)

Prototype: ~2500 lines of JavaScript [1]. API mirrors Automerge [2]:

- `init()` — empty document
- `from(obj)` — populate from JSON object
- `change(doc, f)` — apply mutation f; store δ locally
- `applyChanges(doc, δ)` — compute $X \sqcup X^\delta$
- `getChanges(doc)` — coalesce deltas for broadcast

JavaScript Proxies: Standard JS syntax auto-transformed into delta mutators (e.g. $doc.k_1[0] = \{k_2 : 1\} \rightarrow$ ORMap.apply $\rightarrow$ ORArray.apply $\rightarrow$ MVReg.write).

Delta Communication: Only X^δ broadcast (not full state). Causal anti-entropy [4] retransmits, enforces causal order, merges batches.

No Tombstones / No GC: Causal context naturally retires obsolete dots. Zero garbage collection needed — unlike Automerge (ever-growing history) and Yjs (heuristic GC).

Integer Index Wrapper: `GetUidFromIdx` and `GetPosFromIdx` sort by stable identifiers; pluggable schemes (Logoot, LSEQ).

Evaluation (Section 8)

Setup:

- Intel i7-8565U, empty initial documents
- Metric: **byte-length of encoded state**
- Compared: **DSON**, **Automerge** [2], **Yjs** [3]

- **Map update:** same key updated repeatedly $\rightarrow$ DSON, Yjs: constant $\rightarrow$ Automerge: linear growth
- **Map insert/delete:** repeated insert & delete $\rightarrow$ DSON: constant $\rightarrow$ Automerge, Yjs: **unbounded**
- **Array update:** same index updated $\rightarrow$ DSON: constant $\rightarrow$ Others: growing metadata
- **Array insert/delete (char):** $\rightarrow$ DSON, Yjs: constant
- **Array insert/delete (map):** $\rightarrow$ DSON: constant $\rightarrow$ Yjs: grows
- **Array insert/delete (nested array):** $\rightarrow$ DSON: constant $\rightarrow$ Yjs: grows

Worst-Case Scenario:

- All replicas update all elements concurrently, then synchronize
- Metadata growth: $\mathcal{O}(k^2)$ per element
- Matches theoretical bound; occurs only under extreme contention

Randomized Micro-benchmark:

- 5 replicas, 200 steps, update probability p_u, sync probability p_s
- 10 runs averaged
- **Metadata remains far below worst-case in practice**

Future Work (Section 9)

- **Join Decomposition** [6]: Practical anti-entropy is wasteful; join decomposition yields optimal δ's for DSON CRDTs — major bandwidth savings.
- **Ad-hoc Clients:** Map client dots (c, i) to server dots $(r, l{+}j)$ at upload; server propagates transformed delta — minimizes short-lived client footprint in causal context.
- **Efficient Replica Removal:** Naively removing dots risks causality violations (stale update re-application). Safe cleanup protocol is open future work.
- **Performance Optimization:** DSON is a research prototype; production use requires optimized encoding, faster joins, network-layer integration.
- **Regional Deployment:** Strong consistency within region; DSON coordinates inter-region. Each region = single replica $\Rightarrow$ bounded n.

Key Insights

- **CompDotFun** is the core enabler: roots are permanently deleted (unlike DotMap), making position overwriting and move semantics correct without resurrection.
- **Forest structure** (depth-3 nesting) encodes concurrent positions compactly; $\mathcal{O}(k^2)$ only under adversarial concurrency.
- **No tombstones** $\Rightarrow$ metadata bounded by live document state, not history — fundamental advantage for long-lived documents.
- Causal context compression ($prefix + exceptions$) keeps replica overhead at $\mathcal{O}(n \log n)$ regardless of update count.

Conclusions (Section 10)

- **DSON** is the first δ-based JSON CRDT with composable maps, arrays, and registers under OR semantics.
- Novel **ORArray** supports concurrent insert, update, delete, and **move** without tombstones or garbage collection.
- Metadata bounded by $\mathcal{O}(k^2 D + n \log n)$ — independent of update history; empirically near best-case.
- Formally proven: convergence, OR-semantics correctness, position semantics (Lemmas 5.2–5.4).
- Relieves application developers of manual conflict resolution in distributed document stores.
- Open-source prototype [1] outperforms Automerge and Yjs on all metadata benchmarks.

References

[1] A. Rinberg et al. *DSON – JSON CRDT Using Delta-Mutations.* GitHub, 2021. `github.com/crdt-ibm-research/json-delta-crdt`

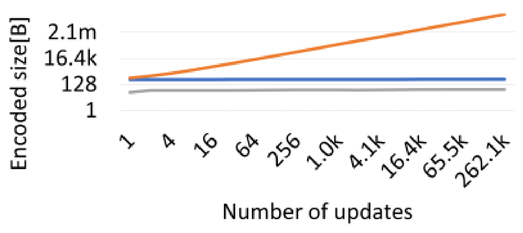
[2] Automerge. *https://github.com/automerge/automerge*, 2021.

[3] Yjs. *https://github.com/yjs/yjs*, 2021.

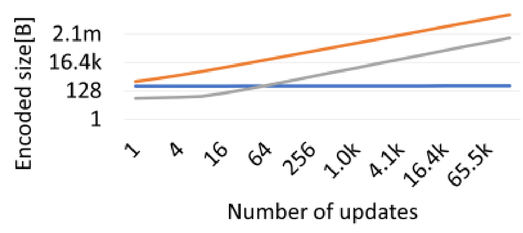
[4] P.S. Almeida, A. Shoker, C. Baquero. *Delta state replicated data types.* J. Parallel Distrib. Comput., 111:162–173, 2018.

[5] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. *Conflict-free replicated data types.* SSS 2011, Springer.

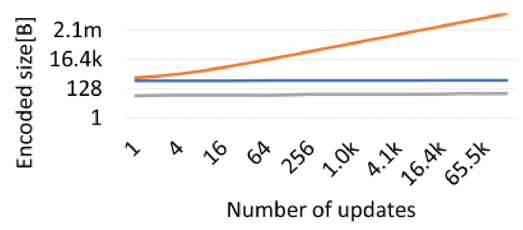
[6] V. Enes, P.S. Almeida, C. Baquero, J. Leitão. *Efficient synchronization of state-based CRDTs.* ICDE 2019, IEEE.



(a) Map size when updating the same key in a map.



(b) Map size when inserting and removing a map key.



(c) Array size when consecutively updating the same index in an array.

—DSON —Automerge —Yjs

Figure 3: Micro benchmarks measuring map and array sizes.